3. Multimodal Visual Understanding + Robotic Arm Gripping (Text Version)

1. Course Content

1. Learn how the multimodal large model integrates 2D visual perception, spatial geometric reasoning, and 3D inverse-kinematics control of the robotic arm;
2. Through terminal text interaction, experience the large model autonomously identifying target objects, computing relative spatial positions, and controlling a 6-DOF robotic arm to perform precise spatial grasping (`pick`);
3. Learn the large model's cross-target relative pose reasoning to complete relative placement (`place`) **Note: because the USB camera has no depth data, the placement location is an adjustable fixed position**;
4. Learn AprilTag-based 3D elevation perception filtering and automated sorting and removal (`apriltag_remove_higher`);
5. Deeply analyze the underlying spatial coordinate solving logic of the embodied large model's decision layer and the action server `action_service_nuwa.py`.
6. **This case uses the matching 3 × 3 × 3 blocks and 3 × 3 × 3 machine-code wooden blocks**

---

2. Preparation

2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Run commands directly in the onboard terminal; there is no need to enter Docker.

2.2 Start the Local Dify Large-Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

The terminal prints the following message, indicating that the Dify service is connected successfully:

```
jetson@yahboom:~$ sh ~/bringup_dify.sh
[+] Running 11/11
 ✔ Container docker-sandbox-1         Started                    1.7s
 ✔ Container docker-web-1             Started                    1.7s
 ✔ Container docker-db-1              Healthy                    2.6s
 ✔ Container docker-redis-1           Started                    1.6s
 ✔ Container docker-weaviate-1        Started                    1.3s
 ✔ Container docker-ssrf_proxy-1      Started                    1.8s
 ✔ Container docker-plugin_daemon-1   Started                    4.2s
 ✔ Container docker-worker_beat-1     Started                    4.5s
 ✔ Container docker-worker-1          Started                    4.5s
 ✔ Container docker-api-1             Started                    4.7s
 ✔ Container docker-nginx-1           Started                    5.7s
```

3. Running Examples

3.1 Start the Embodied Intelligence Master Control Service (Text Interaction Mode)

Open a terminal on the onboard computer and enter the following command to start the large-model embodied control hub (specifying `text_chat_mode:=True`):

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

After the system finishes initializing, the action service and multimodal perception nodes become ready.

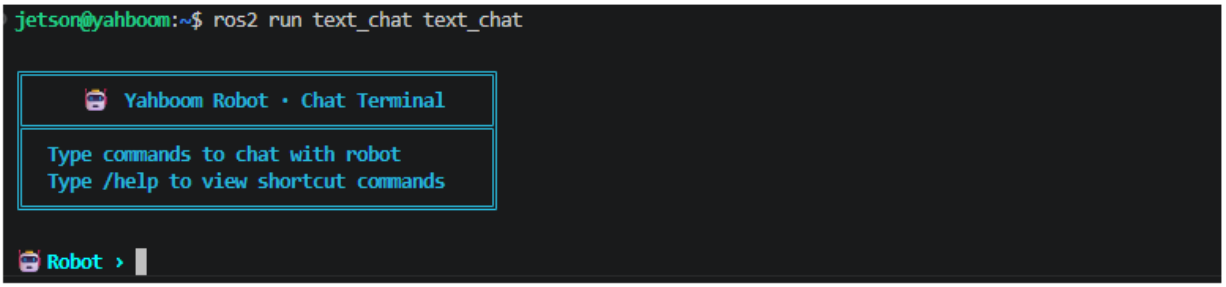
```
Info.cpp] [11] [TfTreeFrameIdInfo] Namespace /ascamera_hp60c
[ros2-1] [ascamera_node-1] [INFO] [1788574077.188951464] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1830] [getConfigurationParameters] according to the configuration file.
[ros2-1] [ascamera_node-1] [INFO] [1788574077.189087142] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1535] [parseConfigFileParameter] moduleName:AS_CAM_HP60C
[ros2-1] [ascamera_node-1] [INFO] [1788574077.189266253] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1562] [parseConfigFileParameter] Parsing config file /home/jetson/yahboomcar_ros2_ws/software/library_ws/src/ascamera/configura
tionfiles/hp60c_v2_01_20241104_configEncrypt.json
[ros2-1] [ascamera_node-1] [INFO] [1788574077.189548715] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1627] [parseConfigFileParameter] configuration protocol Version: v2.x
[ros2-1] [ascamera_node-1] [INFO] [1788574077.189589460] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1638] [parseConfigFileParameter] configuration Version: v2.0.1.20241104
[ros2-1] [ascamera_node-1] [INFO] [1788574077.194612187] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:57[INFO] [CameraHp60c.c
pp] [1803] [setParamersAfterOpenCam] Camera confiParaEnable true, setting configuration parameter
[ros2-1] [ascamera_node-1] [INFO] [1788574078.711008565] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:58[ERROR] [XuCmdCameraH
p60c.cpp] [454] [getMjpegSize] get mjpeg from xu command failed
[ros2-1] [ascamera_node-1] [INFO] [1788574078.711252074] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:58[INFO] [XuCmdCameraHp
60c.cpp] [458] [getMjpegSize] mjpeg size:640x480
[ros2-1] [ascamera_node-1] [INFO] [1788574078.711457367] [ascamera_hp60c.camera_publisher]: camera opened
[ros2-1] [ascamera_node-1] [INFO] [1788574078.711556301] [ascamera_hp60c.camera_publisher]: get config info, ret 0, is_Registration
1
[ros2-1] [ascamera_node-1] [INFO] [1788574078.713432999] [ascamera_hp60c.camera_publisher]: #camera[0xaaaaef410010] SN[ASC60CF190019
84]'s firmware version:DLS_VERc20865=:V01.09_2024110714:V01.19_2024110714:L01
[ros2-1] [ascamera_node-1] [INFO] [1788574078.714586723] [ascamera_hp60c.camera_publisher]: set depth resolution: 640 x 480 @ 25fps
[ros2-1] [ascamera_node-1] [INFO] [1788574078.715638793] [ascamera_hp60c.camera_publisher]: set rgb resolution: 640 x 480 @ 25fps
[ros2-1] [ascamera_node-1] [INFO] [1788574078.723967364] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:58[INFO] [CameraHp60c.c
pp] [259] [startStreaming] start streaming
[ros2-1] [ascamera_node-1] [INFO] [1788574078.724099105] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:07:58[INFO] [CameraSrv.cpp
] [206] [onAttached] attached end
[ros2-1] [ascamera_node-1] [INFO] [1788574080.177118513] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:08:00[INFO] [CameraHp60c.c
pp] [911] [setInternalParameter] mjpeg info: size(640x480)
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715216343] [ascamera_hp60c.camera_publisher]: SN [ ASC60CF19001984 ]'s parameter:
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715495988] [ascamera_hp60c.camera_publisher]: irfx: 435.041
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715544062] [ascamera_hp60c.camera_publisher]: irfy: 434.467
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715571813] [ascamera_hp60c.camera_publisher]: ircx: 317.026
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715595530] [ascamera_hp60c.camera_publisher]: ircy: 237.129
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715625520] [ascamera_hp60c.camera_publisher]: rgbfx: 585.452
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715649590] [ascamera_hp60c.camera_publisher]: rgbfy: 585.031
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715673083] [ascamera_hp60c.camera_publisher]: rgbcx: 321.329
[ros2-1] [ascamera_node-1] [INFO] [1788574080.715698272] [ascamera_hp60c.camera_publisher]: rgbcy: 237.211
[ros2-1] [ascamera_node-1]
[ros2-1] [ascamera_node-1] [INFO] [1788574080.722069877] [ascamera_hp60c.camera_publisher]: publish depth info
[ros2-1] [ascamera_node-1] [INFO] [1788574080.723635692] [ascamera_hp60c.camera_publisher]: publish color(rgb) info
[ros2-1] [ascamera_node-1] [INFO] [1788574080.729896745] [ascamera_hp60c.camera_publisher]: publish tf info
[ros2-1] [ascamera_node-1] [INFO] [1788574080.853537288] [ascamera_hp60c.camera_publisher]: 2026-09-05 10:08:00[INFO] [CameraHp60c.c
pp] [1148] [streamCallback] set gain ret 0, gain 1
[INFO] [model_service-18]: process started with pid [174854]
[INFO] [action_service_nuwa-19]: process started with pid [174856]
[action_service_nuwa-19] [INFO] [1788574085.090039531] [action_service_nuwa]: ROS_Action_Service Initialization completed
[model_service-18] [INFO] [1788574086.521127582] [model_service]: Dify LLM connection successful
[model_service-18] [INFO] [1788574086.522783597] [model_service]: ROS_LLM_Service Initialization completed
```

3.2 Start the Text Interaction Control Terminal

Using **remote SSH**, open another terminal and start the text interaction program:

```
ros2 run text_chat text_chat
```

After startup, the terminal enters the real-time text interaction interface, waiting for the user to enter commands.



> **Tip**
> **Chinese Input Tip**:
>
> - It is recommended to connect from a computer using an SSH remote tool (such as MobaXterm or VSCode Terminal), so that Chinese can be typed seamlessly for interaction.

3.3 Classic Test Cases

> **Note**: RDK and Raspberry Pi boards may experience very laggy VNC when running visualization due to performance issues. The tutorial does not enable the viewing angle by default. You can actually move the cube to track. To enable it, go to the launch file and manually change `display_windows_only_track` to `True`, then compile.

Props recommended for testing:

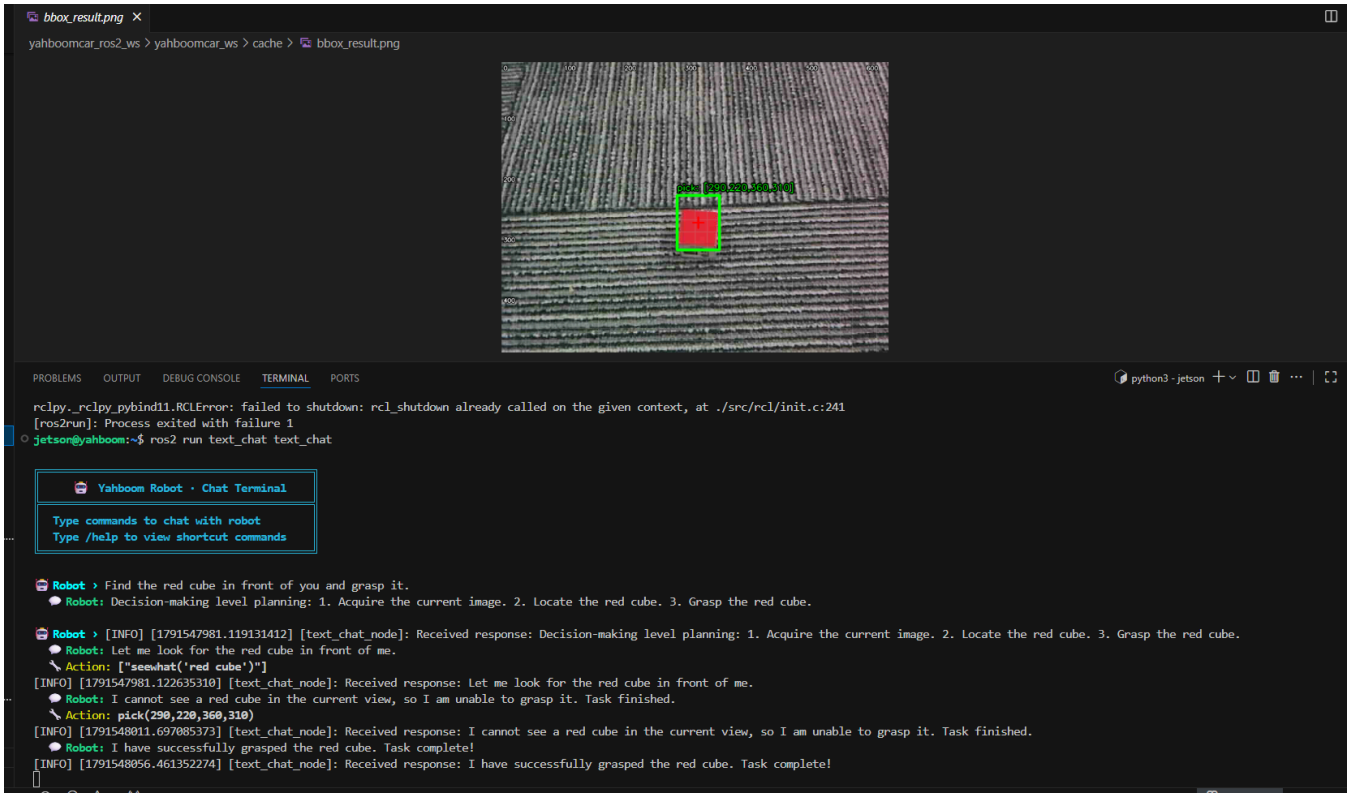
- Basic block test: 30 × 30 × 30 mm building blocks (solid-color blocks such as red, green, and blue);
- Elevation and tag test: 30 × 30 × 30 mm building blocks with AprilTag codes attached (they can be stacked in multiple layers to form different heights).

Case 1: Target Object Recognition and Precise Grasping ("Find the red cube in front of you and grasp it.")

Enter the test statement in the text interaction terminal:

```
Find the red cube in front of you and grasp it.
```

The terminal prints the steps planned by the large model and the bounding box parameters:

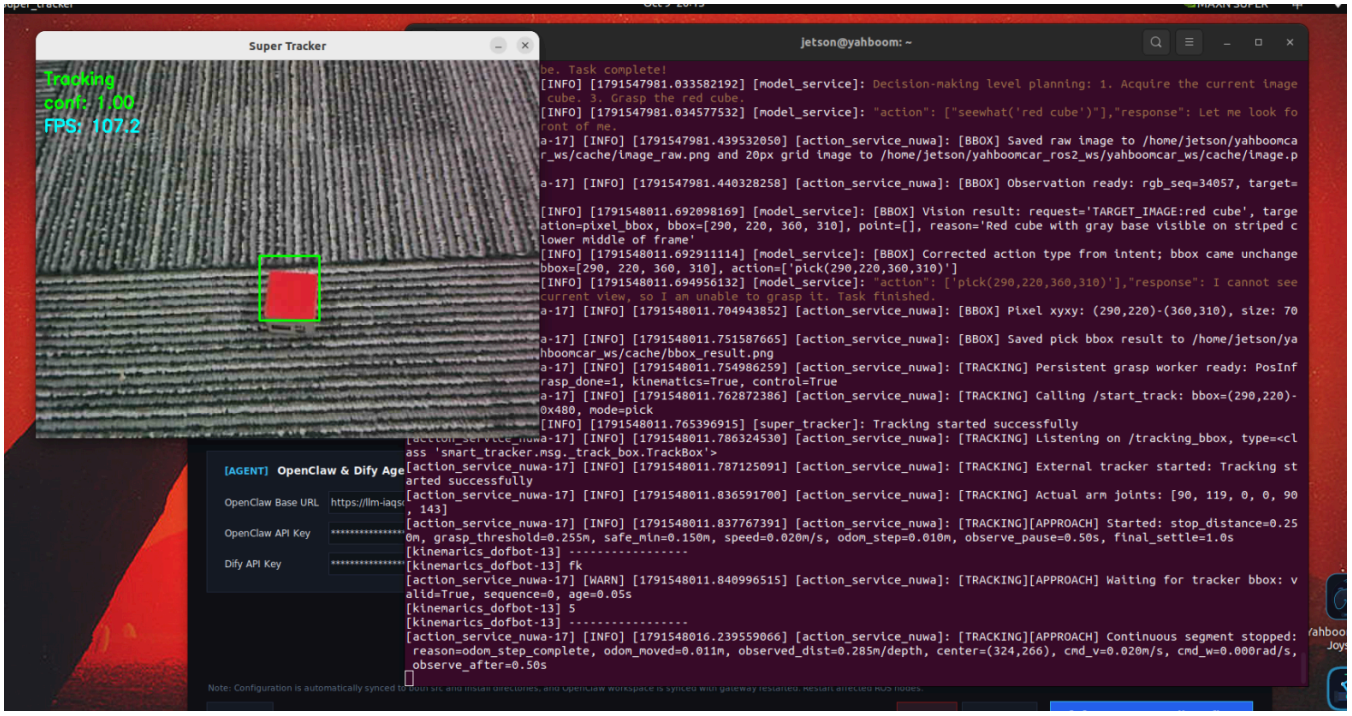


The path of the boxed photo is,

`~/yahboomcar_ros2_ws/yahboomcar_ws/cache/bbox_result.png`

Execution Process and Behavior:

1. After receiving the command, the large model first issues the `seewhat` command to obtain the current color image from the camera;
2. The multimodal model precisely locates the pixel bounding box coordinates `[x1, y1, x2, y2]` of the yellow block and assembles the `pick` action;
3. The robot performs the underlying spatial inverse solution, automatically adjusts the robotic arm angle, opens the gripper, and dives down to grasp the target block;
4. After a successful grasp, the robotic arm lifts up and maintains the current holding posture:



> **Note**
>
> **How to Reset the Robotic Arm**:
>
> - **Method 1**: Enter `put down the block you just grasped` in the interaction terminal, and the robotic arm will safely place the block down in place;
> - **Method 2**: Enter `you can rest now` in the interaction terminal to end the current task cycle and reset the conversation.

Example of entering "you can rest now":

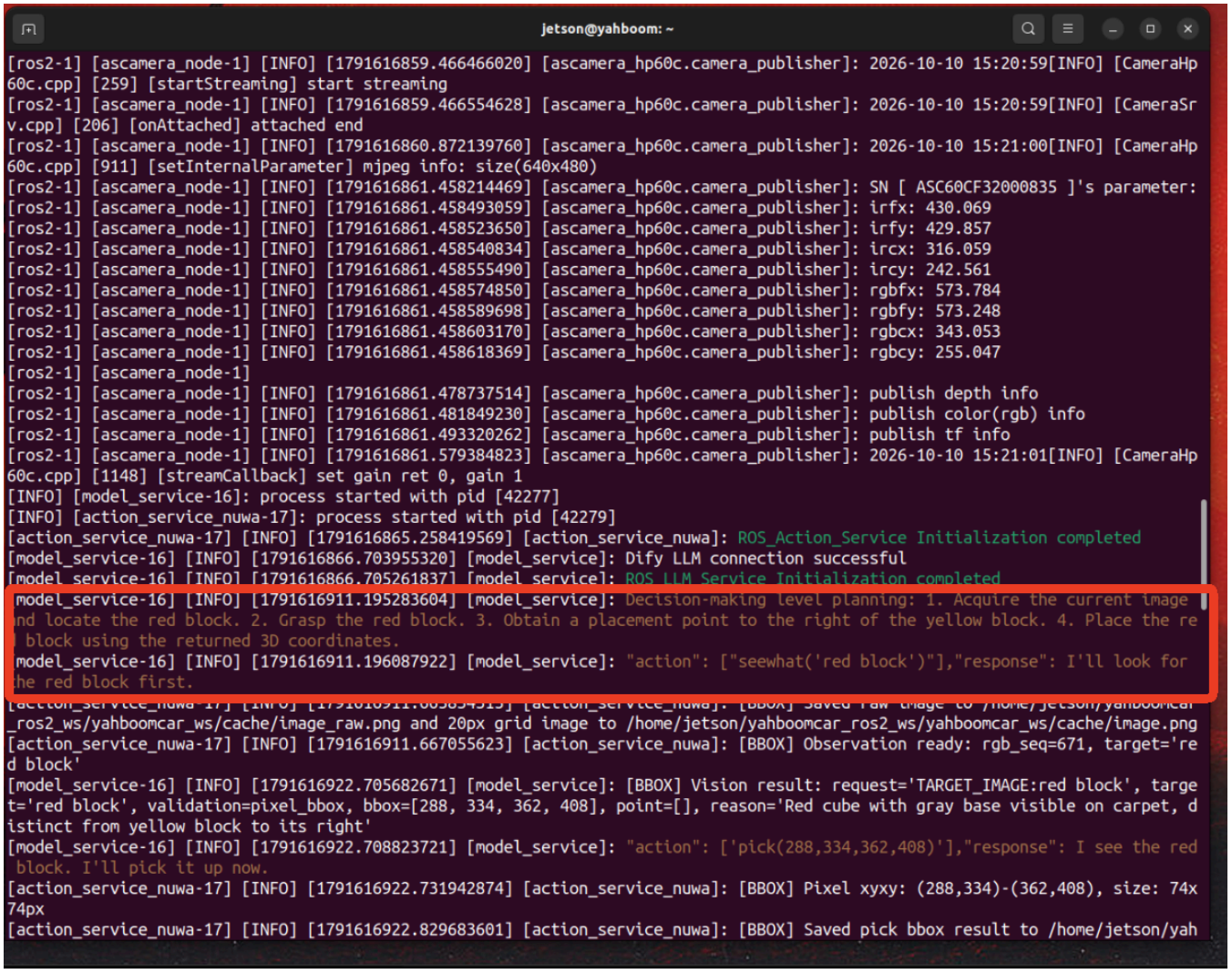


Case 2: Cross-Target Relative Spatial Relationship Placement ("Put the red block in front of you to the right of the yellow block")

Place a red block and a yellow block on the tabletop, and enter the test statement in the terminal:

```
Put the red block in front of you to the right of the yellow block.
```

The terminal prints the 4-stage task plan parsed by the large model:

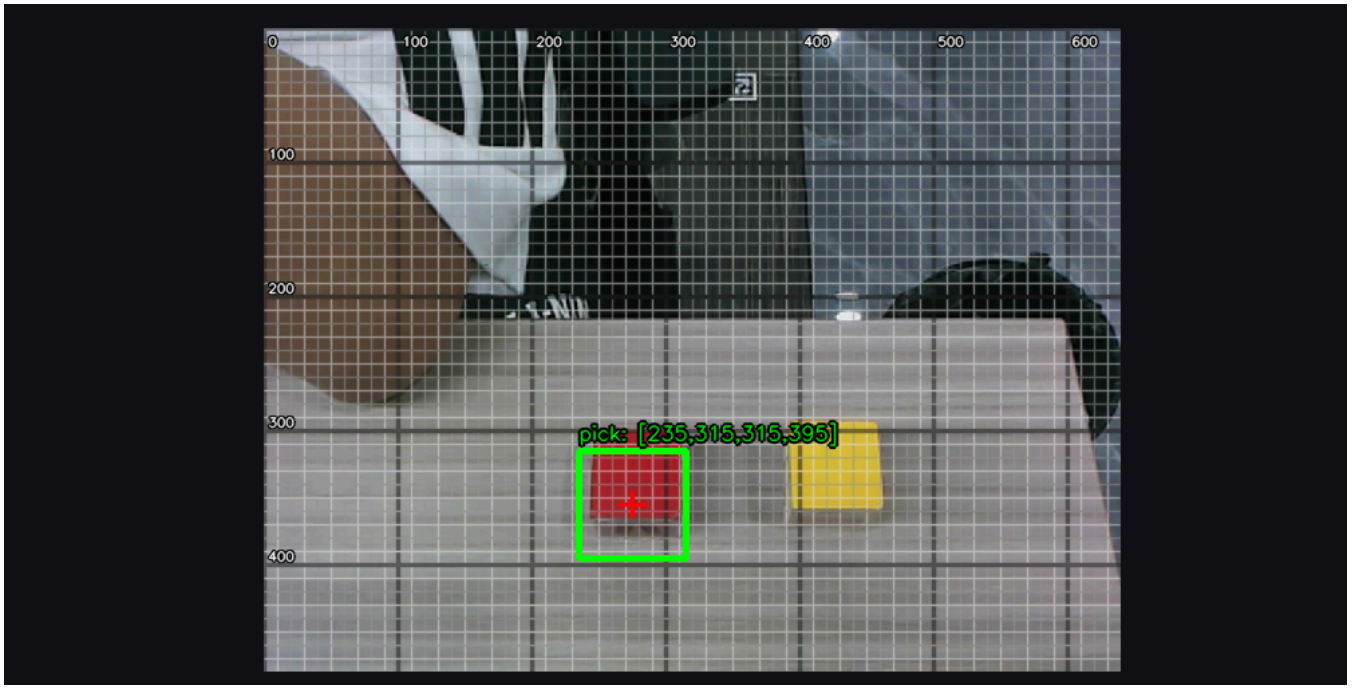


Execution Process and Behavior:

1. The multimodal large model simultaneously detects the pixel coordinates of the "red block" and the "yellow block" in the current field of view;
2. The robot first calls `pick` to grasp the red block in front:

```
jetson@yahboom: ~
the red block first.
[action_service_nuwa-17] [INFO] [1791616911.665834315] [action_service_nuwa]: [BBOX] Saved raw image to /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_raw.png and 20px grid image to /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/cache/image.png
[action_service_nuwa-17] [INFO] [1791616911.667055623] [action_service_nuwa]: [BBOX] Observation ready: rgb_seq=671, target='red block'
[model_service-16] [INFO] [1791616922.705682671] [model_service]: [BBOX] Vision result: request='TARGET_IMAGE:red block', target='red block', validation=pixel_bbox, bbox=[288, 334, 362, 408], point=[], reason='Red cube with gray base visible on carpet, distinct from yellow block to its right'
[model_service-16] [INFO] [1791616922.708823721] [model_service]: "action": ['pick(288,334,362,408)'],"response": I see the red block. I'll pick it up now.
[action_service_nuwa-17] [INFO] [1791616922.731942874] [action_service_nuwa]: [BBOX] Pixel xyxy: (288,334)-(362,408), size: 74x74px
[action_service_nuwa-17] [INFO] [1791616922.829683601] [action_service_nuwa]: [BBOX] Saved pick bbox result to /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/cache/bbox_result.png
[action_service_nuwa-17] [INFO] [1791616922.845923728] [action_service_nuwa]: [TRACKING] Persistent grasp worker ready: PosInfo=1, Curjoints=1, grasp_done=1, kinematics=True, control=True
[action_service_nuwa-17] [INFO] [1791616922.848915338] [action_service_nuwa]: [TRACKING] Calling /start_track: bbox=(288,334)-(362,408), frame=640x480, mode=pick
[super_tracker-11] [INFO] [1791616922.856588410] [super_tracker]: Tracking started successfully
[action_service_nuwa-17] [INFO] [1791616922.872257306] [action_service_nuwa]: [TRACKING] Listening on /tracking_bbox, type=<class 'smart_tracker.msg._track_box.TrackBox'>
[action_service_nuwa-17] [INFO] [1791616922.873096632] [action_service_nuwa]: [TRACKING] External tracker started: Tracking started successfully
[action_service_nuwa-17] [INFO] [1791616922.937680820] [action_service_nuwa]: [TRACKING] Actual arm joints: [90, 120, -1, 0, 90, 30]
[action_service_nuwa-17] [INFO] [1791616922.940323214] [action_service_nuwa]: [TRACKING][APPROACH] Started: stop_distance=0.250m, grasp_threshold=0.255m, safe_min=0.150m, speed=0.020m/s, odom_step=0.010m, observe_pause=0.50s, final_settle=1.0s
[action_service_nuwa-17] [WARN] [1791616922.941262636] [action_service_nuwa]: [TRACKING][APPROACH] Waiting for tracker bbox: valid=True, sequence=0, age=0.07s
[kinemarics_dofbot-13] -----------------
[kinemarics_dofbot-13] fk
[kinemarics_dofbot-13] 5
[kinemarics_dofbot-13] -----------------
[action_service_nuwa-17] [INFO] [1791616923.386653558] [action_service_nuwa]: [TRACKING][GRASP] Range reached; stopping for final 1.0s settle before fresh bbox/depth recheck
[action_service_nuwa-17] [INFO] [1791616924.689572302] [action_service_nuwa]: [TRACKING] Actual arm joints: [90, 120, -1, 0, 90, 30]
[kinemarics_dofbot-13] -----------------
[kinemarics_dofbot-13] fk
[kinemarics_dofbot-13] 5
[kinemarics_dofbot-13] -----------------
[grasp_desktop_remove-15] [INFO] [1791616924.886388725] [multi_brains_grasp_worker]: pos_msg.x: 317.0, pos_msg.y: 366.0, pos_msg.z: 0.2540000081062317
```

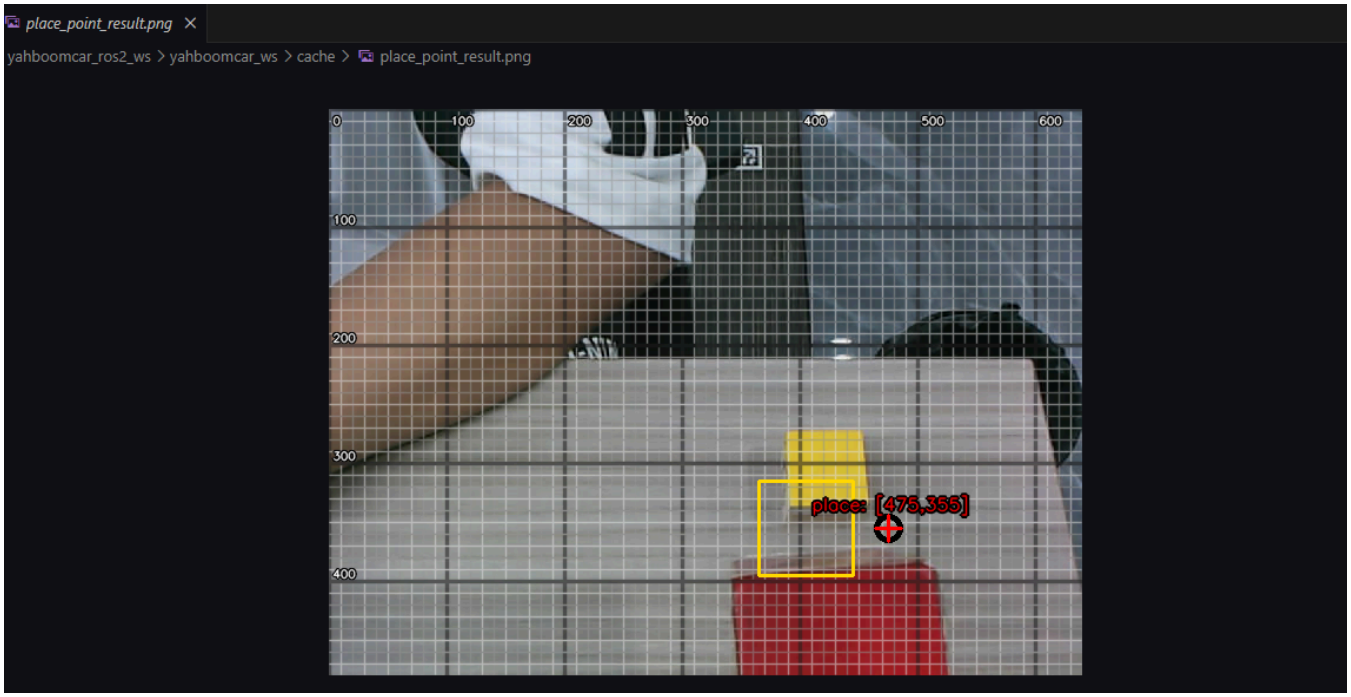
```
~/yahboomcar_ros2_ws/yahboomcar_ws/cache/bbox_result.png
```



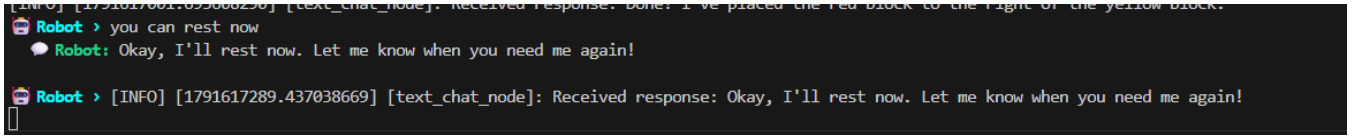
3. Then, based on the coordinates of the blue block and the geometric topological relationship of "to the right," the large model calls the `place` command; the robotic arm rotates to the right side of the base and safely releases the gripper, completing a precise placement and indicating that the task is complete:

```
jetson@yahboom: ~
6.5/30.0s
[action_service_nuwa-17] [INFO] [1791616931.691585196] [action_service_nuwa]: [TRACKING] Waiting for grasp completion: elapsed=
7.0/30.0s
[action_service_nuwa-17] [INFO] [1791616932.198790829] [action_service_nuwa]: [TRACKING] Waiting for grasp completion: elapsed=
7.5/30.0s
[grasp_desktop_remove-15] [INFO] [1791616932.475528197] [multi_brains_grasp_worker]: Straightening chassis wheels: linear.x=-0.
4, angular.z=0.0
[grasp_desktop_remove-15] [INFO] [1791616932.578270548] [multi_brains_grasp_worker]: Grasp cycle completed
[action_service_nuwa-17] [INFO] [1791616932.600898804] [action_service_nuwa]: [TRACKING] Object grasp completed
[action_service_nuwa-17] [INFO] [1791616932.623934324] [action_service_nuwa]: [GRASP] pick grasp completed and secured at joint
6=143; settling for 1.0s
[model_service-16] [INFO] [1791616943.241764760] [model_service]: "action": ["GetPlacePoint('to the right of the yellow block')
"],"response": Red block grasped. Now finding a spot to the right of the yellow block.
[action_service_nuwa-17] [INFO] [1791616943.407204081] [action_service_nuwa]: [BBOX] Saved raw image to /home/jetson/yahboomcar
_ros2_ws/yahboomcar_ws/cache/image_raw.png and 20px grid image to /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/cache/image.png
[model_service-16] [INFO] [1791616963.546973009] [model_service]: [BBOX] Vision result: request='PLACE_POINT_IMAGE:to the right
 of the yellow block', target='placement point to the right of the yellow block', validation=pixel_point, bbox=[], point=[520,
350], reason='Yellow block visible at approximately x=450-500, y=300-400; selected point at x=520 is clearly to the right with
adequate clearance on carpet surface'
[model_service-16] [INFO] [1791616963.549632017] [model_service]: "action": ['GetPlacePoint(520,350)'],"response": I see the ye
llow block. I'll get the exact placement coordinates to its right
[action_service_nuwa-17] [INFO] [1791616963.826919993] [action_service_nuwa]: [PLACE] Using fresh placement sensors: rgb_seq=11
15, depth_seq=12
[pcl_segment-12] [INFO] [1791616964.012066655] [pcl_segment]: Target pose in base_link: (0.226, -0.087, 0.014)
[pcl_segment-12] [INFO] [1791616964.012220927] [pcl_segment]: Circle ROI: center=(520,350), radius=10, avg_depth=0.254m, target
_pose=(0.226, -0.087, 0.014)
[pcl_segment-12] [INFO] [1791616964.012254559] [pcl_segment]: GetTargetPose service completed successfully.
[action_service_nuwa-17] [INFO] [1791616964.078753443] [action_service_nuwa]: [PLACE] Saved place point result to /home/jetson/
yahboomcar_ros2_ws/yahboomcar_ws/cache/place_point_result.png
[action_service_nuwa-17] [INFO] [1791616964.079545763] [action_service_nuwa]: [PLACE] Located validated point: pixel=(520,350),
 base_link=(0.226,-0.087,0.014)
[model_service-16] [INFO] [1791616977.094142955] [model_service]: "action": ['place(0.2260,-0.0871,0.0137)'],"response": Placin
g the red block to the right of the yellow block now.
[action_service_nuwa-17] [WARN] [1791616977.101901807] [action_service_nuwa]: [TRACKING] Failed to read actual arm joints: no r
esponse; using the configured fallback pose
[kinematics_dofbot-13] -----------------
[kinematics_dofbot-13] ik
[kinematics_dofbot-13] 0.186427-0.08711540.0637456
[kinematics_dofbot-13] ik seed: 90,120,0,0,90,
[kinematics_dofbot-13] +-+-+-+
[kinematics_dofbot-13] [    0.267499,          0,    0.963558;
[kinematics_dofbot-13]            0,          1,           0;
[kinematics_dofbot-13]     -0.963558,          0,    0.267499]
```

~/yahboomcar_ros2_ws/yahboomcar_ws/cache/place_point_result.png~



4. Enter "you can rest now" to reset the robotic arm and start a new round of interaction:

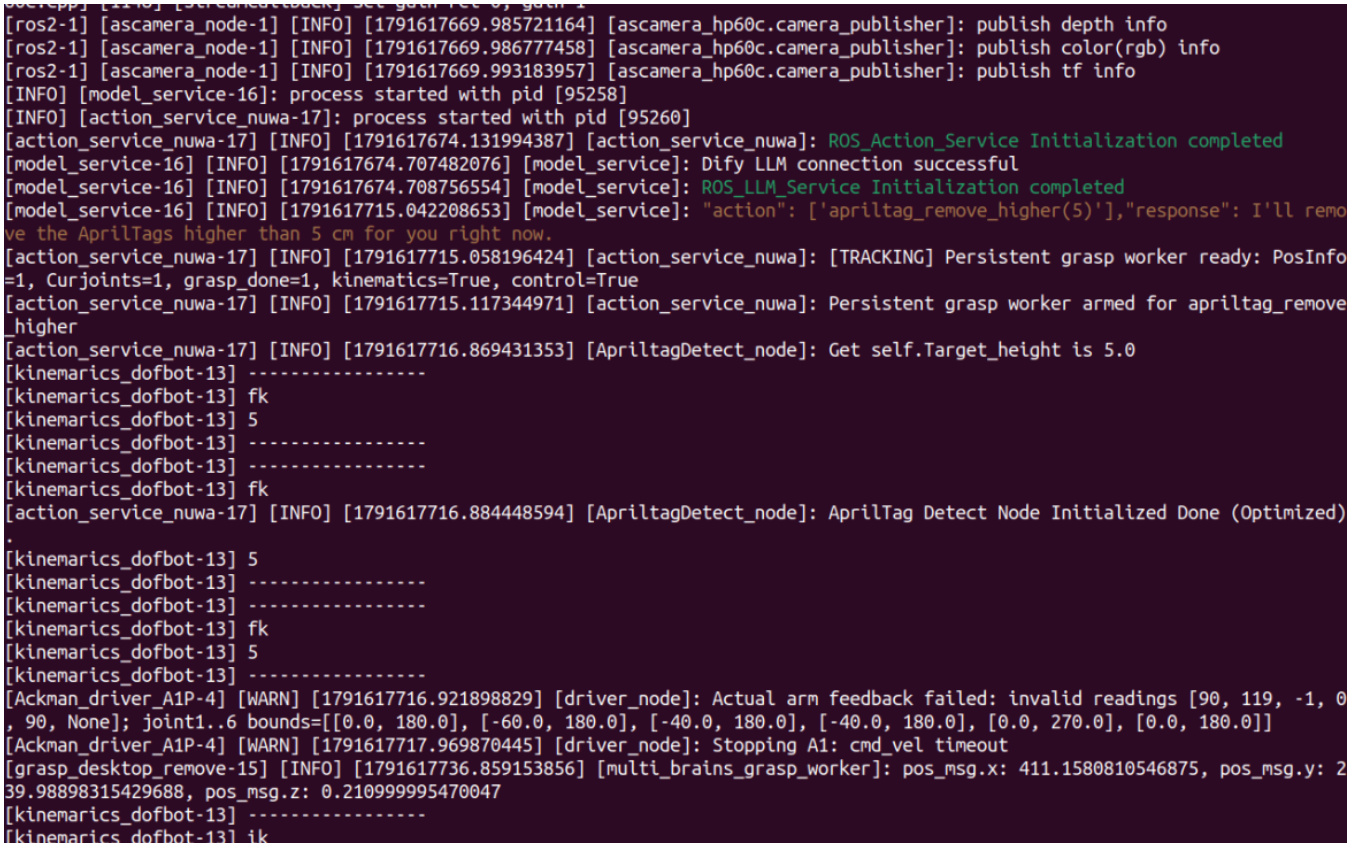


Case 3: 3D Elevation Perception and Automated Sorting/Removal ("Please help me remove the machine codes in front of you that are higher than 5 cm")

Place several wooden blocks with AprilTag tags in the field of view, and raise one of them to a height of more than 5 cm. Enter in the interaction terminal:

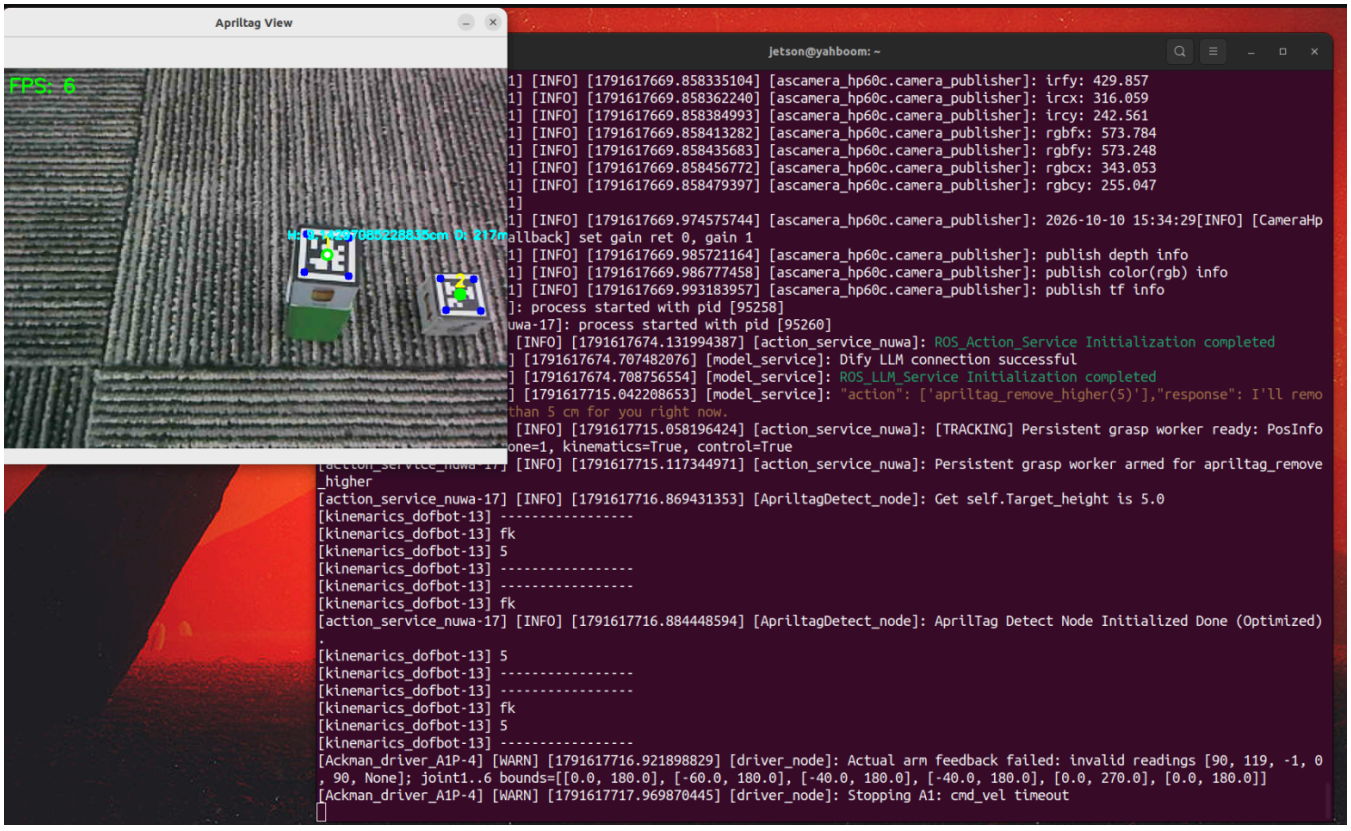
```
Please help me remove the machine codes in front of you that are higher than 5 cm.
```

The terminal prints the `apriltag_remove_higher` action parsed by the large model:

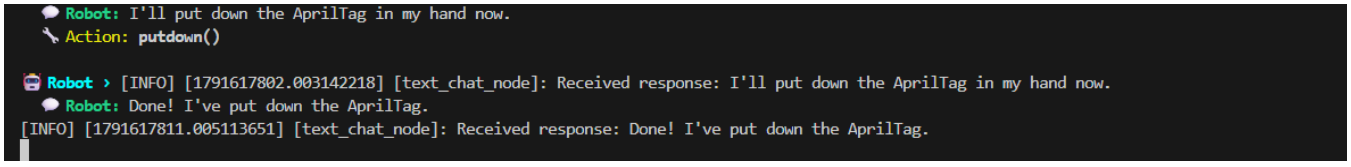


Execution Process and Behavior:

1. The robot uses the camera to perceive the pose and spatial height of AprilTag tags, and the window annotates the 3D relative height of each tag in real time;
2. When a tag with a height exceeding 5 cm (0.05 m) is detected, the robotic arm automatically locates its spatial center and reaches down to grasp it:



To put it down, you can say "put down the machine code in your hand"



4. Core Source Code Analysis

Source package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

For different hardware configurations, the system provides two independent action execution servers and adaptively switches between them via the environment variable `CAMERA_TYPE` (`nuwa` or `usb`):

- **3D depth camera scheme** (default): provided by `action_service_nuwa.py`, which uses point clouds and depth maps to achieve back-projection inverse-solution grasping in physical space;
- **USB monocular camera scheme**: provided by `action_service_usb.py`, which combines Visual Servoing with KCF tracking for closed-loop approach grasping.

4.1 Scheme 1: 3D Depth Camera Spatial Inverse-Solution Grasping (action_service_nuwa.py)

In the depth camera scheme, a pixel grid is overlaid on the image captured by the camera, and the multimodal large model outputs a 2D bounding box `[x1, y1, x2, y2]`:

In the `ActionController.pick()` method, the process of solving from 2D pixels to the real physical coordinates of the robotic arm is completed:

```
def pick(self, x1, y1, x2, y2):
    # 1. Normalize the pixel bounding box to prevent coordinates from going out of bounds
    x1, y1, x2, y2 = self.normalize_bbox(x1, y1, x2, y2, 640, 480)

    # 2. Extract the valid foreground distance Z of the target bounding box in the depth map
    depth = self.get_dist(int((x1 + x2) / 2.0), int((y1 + y2) / 2.0))

    # 3. Combined with the camera intrinsics, back-project to a 3D point in the camera coordinate system (cam_x, cam_y, cam_z)
    point_cam = self.pixel_to_3d_ray(u_c, v_c, depth)

    # 4. Use the TF transform to convert the camera point to the robotic arm base coordinate system (arm_base_link)
    point_arm = self.tf_buffer.transform(point_cam, "arm_base_link")

    # 5. Call the 6-DOF robotic arm inverse kinematics (IK) to solve the target servo angle of each joint
    target_angles = self.solve_ik(point_arm.point.x, point_arm.point.y, point_arm.point.z)

    # 6. Safely drive the servo to reach down and close the gripper
    self.pubSix_Arm(target_angles, runtime=2000)
    self.pubSingle_Arm(joint_id=6, joint_angle=30, runtime=1000)  # Close the gripper
```

Spatial Relative Placement (place)

When the large model understands "place it to the right/left of an object," the decision hub first generates the target placement point `(place_x, place_y, place_z)` based on the reference target's BBox and the relative direction, and `ActionController.place()` executes it:

1. The robotic arm lifts the grasped object to a safe transfer height;
2. The base rotates to the lateral unloading point of the target direction;
3. It slowly descends and opens the gripper to release the object;
4. The robotic arm withdraws to the standby pose, ensuring it does not scrape or bump the placed item.

4.2 Scheme 2: USB Monocular Camera Visual-Servoing Grasping (action_service_usb.py)

A monocular USB camera cannot directly read the 3D physical depth of a pixel. Therefore, `action_service_usb.py` adopts a **closed-loop guided grasping scheme based on Visual Servoing**:

```
def grasp_obj(self, x1, y1, x2, y2) -> bool | None:
    """ Start tracking and grasp the target according to the 640x480 BBox given by the large model """
    normalized = self.normalize_bbox(x1, y1, x2, y2)
    if normalized is None:
        self.get_logger().error("[BBOX] Failed to normalize the bounding box")
        return False
    x1, y1, x2, y2 = normalized
    self.save_bbox_result(normalized, "grasp_obj")
```

```
    # Start the visual detection and servo tracking sub-node
    self.grasp_obj_future = Future()
    command = [
        "ros2", "run", "dofbot_pro_usb_voice", "KCF_detect",
        "--ros-args",
        "-p", f"x1:={x1}",
        "-p", f"y1:={y1}",
        "-p", f"y2:={y2}",
    ]
    return self._run_usb_grasp_action(
        "grasp_obj",
        self.grasp_obj_future,
        command,
        cancel_after=False,
    )
```

- **Closed-loop servoing flow**:
    1. After the system receives the object pixel box returned by the large model, it starts the `KCF_detect` visual tracking node to lock onto the target;
    2. Under closed-loop visual feedback control, the robotic arm and the robot chassis dynamically fine-tune their pose and distance, keeping the target object centered while approaching the optimal grasping working range of the robotic arm;
    3. Once ranging and alignment converge, the grasping action sequence is triggered to reach down and close the gripper, achieving highly robust grasping with a monocular camera.

Spatial Relative Placement (place)

A monocular USB camera cannot directly read the 3D physical depth of a pixel, so the placement location is an adjustable fixed position:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

The following are the adjustable left/right placement positions:

```
self.right_putsown_joints = [130,20,55,50,90,135]  # Robot arm put down right pose
self.left_putsown_joints = [30,20,55,50,90,135]
```

After modifying, you need to compile and then restart the program

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
colcon build --packages-select multi_brains
```

4.3 Tag Elevation Perception and Sorting (apriltag_remove_higher)

Both schemes support the tag recognition and sorting action `apriltag_remove_higher(target_high)`:

- Traverse all detected AprilTag machine-code tags in the field of view;
- Extract the vertical height of each tag relative to the reference plane;
- When a tag whose height exceeds the set threshold (such as `0.05m`) is detected, automatically trigger

spatially aligned grasping and move it to the side, realizing intelligent visual elevation sorting.